Engineering AI Marketplace

1. Purpose and scope

Teams publish useful AI helpers, but other teams struggle to discover them, assess their quality, or adopt them safely. The marketplace turns these helpers into **versioned, supported engineering capabilities**. Success means less time to an accepted engineering result, with quality maintained or improved and security as a condition of adoption. Cross-team reuse multiplies the benefit.

Engineering teams contribute and maintain assets. The platform team owns packaging standards, shared connectors, execution infrastructure, and policy. Domain reviewers approve higher-risk uses. The marketplace complements existing tools; it does not replace their authorization or engineering sign-off.

Assumptions: corporate identity is available; the pilot handles public/internal data through approved enterprise model routes. Restricted or unclassified data requires a separate policy and is excluded initially. Assets execute in managed environments, including workers near lab and simulation resources. Engineers receive access to services rather than executable downloads.

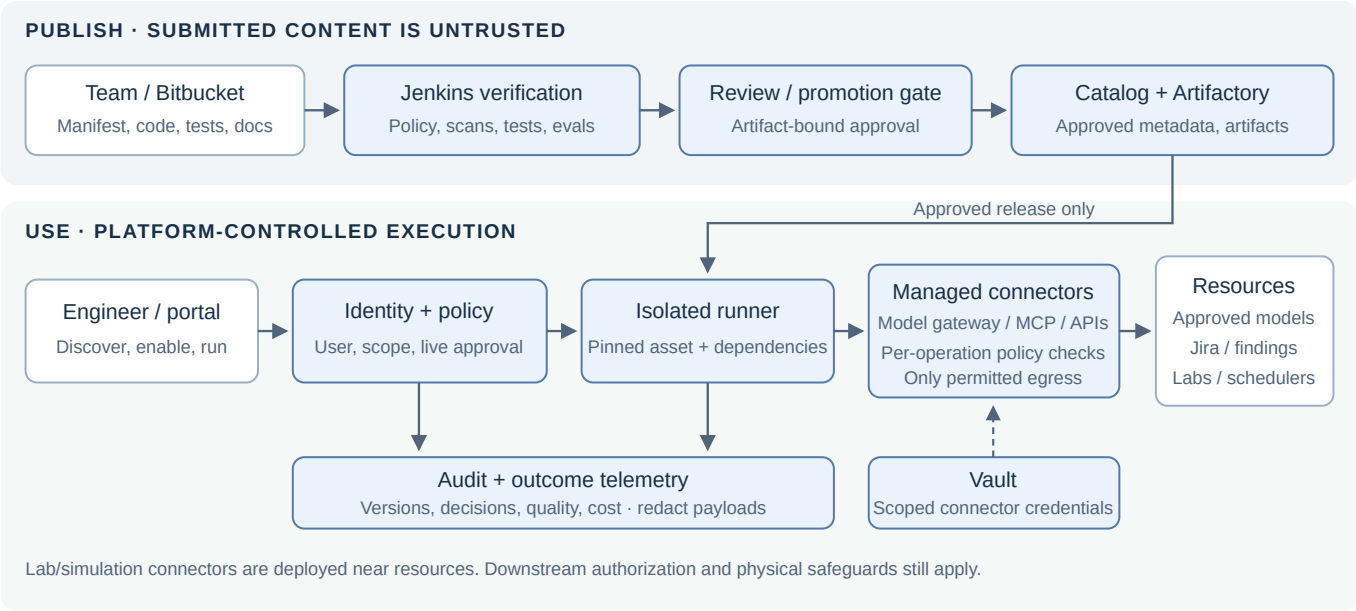
2. Assets and reusable building blocks

Asset	Purpose and users	Key control
Jira test-plan agent	Software/firmware engineers obtain a draft mapped to story acceptance criteria. Start with a fixed workflow; add adaptive retrieval only if it improves evaluated results.	Read authorized stories; engineer reviews assumptions and coverage.
Security-triage skill	Instructions, references, and optional scripts help all teams assess findings and propose remediation. Loaded by a managed runner, not executed independently.	Preserve machine recommendations and human decisions separately; no automatic dismissal.
Lab-instrument MCP server	Hardware engineers access bounded measurement and instrument operations.	Equipment authorization, exclusive access where needed, command limits, and independent physical interlocks.
Simulation MCP server	Simulation engineers submit bounded jobs, retrieve results, and obtain summaries through a hosted AI workflow.	Compute/license budgets, job ownership, cancellation, and traceability to parameters and raw results.
Evaluation pack	Authors, QA, and reviewers share sanitized cases, expert-reviewed criteria, and scoring code as a common baseline across assets and versions.	Review fixtures and scorer; version the pack and evaluation configuration. Passing is evidence, not a guarantee.

An agent coordinates work; a skill supplies a procedure; an MCP server exposes capabilities. An agent can load a reviewed skill and call approved tools. Every executable component and dependency must belong to the approved release.

Evaluations may live in an asset's package; publish shared packs separately when reuse or independent QA ownership helps. Jira test-plan agents can pin the same pack to compare coverage and invented requirements across agents and releases, adding local cases without weakening shared criteria. When a baseline changes, re-evaluate both current and candidate releases against it.

3. Architecture and existing-tool integration



One platform, two paths: verified publication and controlled execution.

Publishing: Bitbucket stores submissions and reviews. Jenkins builds candidates into restricted Artifactory staging and runs manifest checks, tests, evaluations, and applicable Black Duck/Polaris scans. Only the platform promotion service can promote the exact reviewed artifact digest to the approved repository and activate its catalog entry. The portal lists approved catalog entries; uploads alone create no listing. Production runners cannot read staging. Confluence holds guides; Jira tracks reviews, incidents, and retirement. Failed or unavailable required checks block promotion.

Execution: the catalog holds metadata and approvals, not engineering payloads. Corporate identity authenticates engineers; isolated runners check live approval and execute pinned releases. Connectors validate user and workload identity, enforce destinations, scopes, and budgets, and hold downstream Vault credentials. Lab/simulation servers run near resources with equivalent controls. Network policy prevents connector bypass. MCP provides integration, not automatic safety; ordinary APIs are also supported.

4. Engineer experience, access, and secrets

1. **Discover:** search by task, domain, owner, compatibility, permitted data, and approval/evaluation evidence. Catalog visibility respects access restrictions.
2. **Enable:** inspect permissions, examples, limitations, and support; enable an approved version in a managed workspace. This replaces local installation. Approved IDE clients may later connect remotely.
3. **Run:** submit a Jira story ID. The connector checks access and supplies only required, permitted fields of that story to the approved model route. Review the draft's acceptance-criteria references, assumptions, and gaps before adoption.

Effective access intersects **user rights, approved asset scope, current policy, and resources needed for the run**, including field and classification limits. Connectors check each sensitive operation and downstream authorization; a service account's broader rights never replace user checks. Resource owners approve connector scopes and credential mappings; manifest requests grant no access. Prefer delegated user access or short-lived, scoped Vault credentials where supported; rotate static credentials separately. Keep downstream credentials with connectors, out of asset packages, model context, and logs. Minimize/redact telemetry and limit retention. Resource content and model outputs are untrusted; prompts cannot override access checks.

5. Manifest contract

The versioned JSON manifest declares an asset's requirements; it is not approval or proof of behavior. The pilot uses a small common core:

Field	Declaration
`schema_version`, `id`, `kind`, `version`	Supported schema; stable ID; agent, skill, MCP server, or evaluation pack; explicit release version.
`owner`	Accountable team and support contact. Production checks ownership against corporate records.
`artifact`	Immutable Artifactory reference and SHA-256 digest. Release evidence binds source revision and dependency inventory.
`execution`	Managed execution profile. Skills and evaluation packs are consumed by a compatible managed runner.
`permissions`	Requested capabilities and explicit resource scopes; no blanket grants. Empty means no resource access; declarations grant none.
`data`	Maximum permitted classification and model-route alias, or no model use. Unknown classification does not imply permission.
`secret_refs`	Logical connector credential references only, never secret values. Requests do not confer credential access.

Production profiles add compatibility, evaluations, network destinations, budgets, and retention. Trusted configuration resolves profiles, model routes, and secret references. Publishers cannot choose arbitrary provider endpoints or self-declare approval. Runtime classification checks prevent publisher labels downgrading sensitive data.

6. Publishing, trust, and lifecycle

1. **Submit:** provide a supported package, manifest, guide, tests, and evaluations. Managed team experiments retain existing access limits and are not approved catalog releases.
2. **Verify:** platform-controlled Jenkins gates check policy, provenance, security/license evidence, tests, and AI quality. Unsupported scan coverage requires review, never silent passing. Evaluate missed cases, invented requirements, and malicious story content.
3. **Review:** assess evidence and access. Security, lab, and simulation owners review domain-specific effects and quotas. Contributors cannot self-approve high-impact releases. Generated text cannot authorize deployment, finding dismissal, or instrument actions.
4. **Approve:** bind reviewer, policy version, artifact digest, configuration/dependencies, permitted uses, and expiry. Changes to permissions, code, dependencies, prompts/skills, or models require relevant re-evaluation. Rollback targets must remain approved.
5. **Maintain or retire:** owners handle incidents/reviews. Deprecation provides migration guidance; expiry/retirement blocks new runs. Revocation disables releases, interrupts managed jobs where possible, and revokes access. Connectors check live approval. Cancel external jobs through schedulers; require safe lab stops. Completed actions and returned data cannot be undone.

7. Value, pilot, and continued investment

Measure	How it informs decisions
Time to accepted result	Include execution, engineer review, and correction time; compare similar tasks with a baseline.
Quality	Expert-reviewed coverage, missed cases, unsupported claims, and downstream rework. Speed gains require quality to remain acceptable.
Reuse	Repeat use and accepted outcomes across distinct teams; demonstrate benefit beyond the author's team.
Cost and reliability	Cost per accepted result, latency, failures, and maintenance effort; track denied operations and incidents separately.

Pilot Jira test planning and read-only security triage across a few teams. Evaluate sanitized benchmarks and representative tasks; record asset and pack versions, local cases, and model configuration. Agree quality thresholds with domain owners; treat time-saving targets as hypotheses. Use task-level aggregates, not developer rankings. Avoid collecting raw stories/prompts merely for analytics.

Adoption: the platform team coordinates representatives, onboarding, Confluence links, demos, and targeted notices after catalog activation. Task, audience, prerequisite, and evidence metadata guide matching, which may be agentic: an agent uses authorized team profiles and stated needs to explain which teams could benefit, respecting catalog visibility. Owners review outreach recommendations; subscriptions and digests limit noise. Measure reach, trials, accepted results, and repeat cross-team use.

Expand to instruments and simulation after validating domain controls. Keep assets with sustained benefit and active owners; improve fixable gaps; retire poor quality, excessive cost, unresolved risk, or unsupported releases. Shared evaluation packs enable repeatable comparisons; one benchmark cannot cover every domain.

8. Working piece: simple manifest validator

A local Python CLI checks **JSON structure and trusted pilot policy**, separate from the manifest. It needs no live integrations and never executes submissions or follows supplied URLs.

- **Structure:** require core fields, correct types, nonblank identifiers/ownership, supported schema/kinds, explicit versions, and valid artifact references/digests. Reject unknown fields and duplicate JSON keys.
- **Execution and access:** require a recognized managed profile, approved artifact location, and explicit resource scopes. The pilot policy permits only named read-only Jira and security-finding capabilities with configured scopes; reject wildcard grants and unsupported capabilities.
- **Data and credentials:** accept only pilot-supported classifications and compatible model-route aliases. Reject unknown/restricted data, arbitrary model destinations, embedded credential fields, and unrecognized secret references. Detecting all secrets hidden in arbitrary strings is outside this check.
- **Result:** return actionable field paths and error codes, with exit 0 for a valid manifest and nonzero for rejection or input failure. Do not label success as production approval.
- **Tests/demo:** include valid Jira, security-skill, and evaluation-pack manifests. Test missing/wrong fields, malformed/duplicate JSON, broad access, unmanaged execution, invalid artifacts, prohibited data/model combinations, and embedded credentials. Reject MCP permissions outside pilot policy pending future profiles.

Limits and tradeoffs: static validation cannot verify artifact contents, truthful declarations, user access, scan authenticity, or AI quality. Promotion and runtime controls supply those checks. Central execution enables enforcement/revocation but adds availability and operating costs; sensitive operations fail closed if authorization or approval is unavailable. The demo implements validation, not the portal or runtime.